

VLSI INTERNSHIP – TASK 3

VERILOG HDL IMPLEMENTATION OF SEQUENTIAL CIRCUITS

Submitted By : Likhith Gowda H R

Department of Electronics and Communication Engineering

Dayananda Sagar Academy of Technology and Management (DSATM)

Internship Domain : VLSI Design

Organization : Maincrafts Technology

July 2026

1. Introduction

Verilog HDL (Hardware Description Language) is widely used for designing and verifying digital circuits in FPGA and ASIC design flows. Unlike combinational circuits, sequential circuits depend on both present inputs and previous states because they incorporate memory elements such as flip-flops and registers.

In this task, sequential circuits including a D Flip-Flop, JK Flip-Flop, 4-bit Register, and 4-bit Binary Counter were implemented using Verilog HDL. Functional verification was carried out using Icarus Verilog, and the generated waveforms were analyzed using GTKWave to validate the correctness of each design.

1.1 Objectives

- To understand sequential circuit design using Verilog HDL.
- To implement D Flip-Flop.
- To implement JK Flip-Flop.
- To implement a 4-bit Register.
- To implement a 4-bit Binary Counter.
- To simulate circuits using Icarus Verilog.
- To analyze waveforms using GTKWave.

1.2 Tools Used

- Verilog HDL
- Icarus Verilog
- GTKWave
- Ubuntu (WSL)

2. D Flip-Flop Implementation

2.1 Verilog Code

The following Verilog HDL code implements a positive-edge-triggered D Flip-Flop.

d_flipflop.v

```
module d_flipflop(  
  
    input clk,  
  
    input D,  
  
    output reg Q  
  
);  
  
always @(posedge clk)  
  
begin  
  
    Q <= D;  
  
end  
  
endmodule
```

2.2 Testbench Code

The following testbench verifies the operation of the D Flip-Flop for different input values.

tb_d_flipflop.v

```
`timescale 1ns/1ps

module tb_d_flipflop;

reg clk;

reg D;

wire Q;

d_flipflop uut (
    .clk(clk),
    .D(D),
    .Q(Q)
);

always #5 clk = ~clk;

initial
begin
    $dumpfile("d_flipflop.vcd");
    $dumpvars(0, tb_d_flipflop);

    clk = 0;

    D = 0;

    #10 D = 1;

    #10 D = 0;

    #10 D = 1;

    #10 D = 1;
```

```

        #10 D = 0;

        #10;

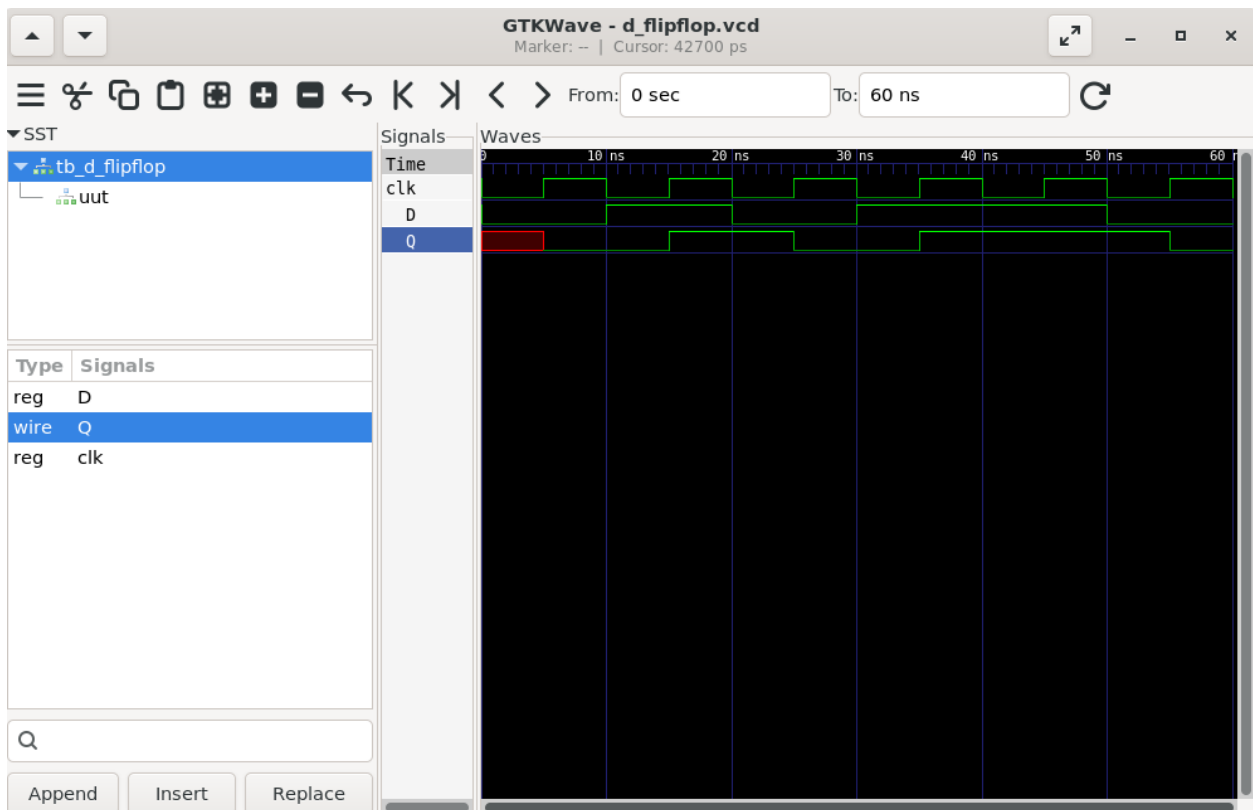
        $finish;

    end

endmodule

```

2.3 Waveform Output



Observation:

The D Flip-Flop successfully captures the input data on every positive edge of the clock. The waveform confirms that the output follows the input only during the rising edge, verifying correct sequential behavior.

3. JK Flip-Flop Implementation

3.1 Verilog Code

The following Verilog HDL code implements a positive-edge-triggered JK Flip-Flop.

jk_flipflop.v

```
module jk_flipflop(  
    input clk,  
    input J,  
    input K,  
    output reg Q  
);  
  
always @(posedge clk)  
begin  
    case ({J, K})  
        2'b00: Q <= Q;  
        2'b01: Q <= 1'b0;  
        2'b10: Q <= 1'b1;  
        2'b11: Q <= ~Q;  
    endcase  
end  
  
endmodule
```

3.2 Testbench Code

The following testbench was developed to verify the Hold, Reset, Set, and Toggle operations of the JK Flip-Flop using different combinations of J and K inputs.

tb_jk_flipflop.v

```
`timescale 1ns/1ps
module tb_jk_flipflop;
    reg clk;
    reg J;
    reg K;
    wire Q;
    jk_flipflop uut (
        .clk(clk),
        .J(J),
        .K(K),
        .Q(Q)
    );
    always #5 clk = ~clk;
    initial
    begin
        $dumpfile("jk_flipflop.vcd");
        $dumpvars(0, tb_jk_flipflop);
        clk = 0;
        J = 0;
        K = 1;
        #10 J = 0; K = 0;
        #10 J = 0; K = 1;
        #10 J = 1; K = 0;
        #10 J = 1; K = 1;
        #10 J = 0; K = 0;
    end
endmodule
```

```

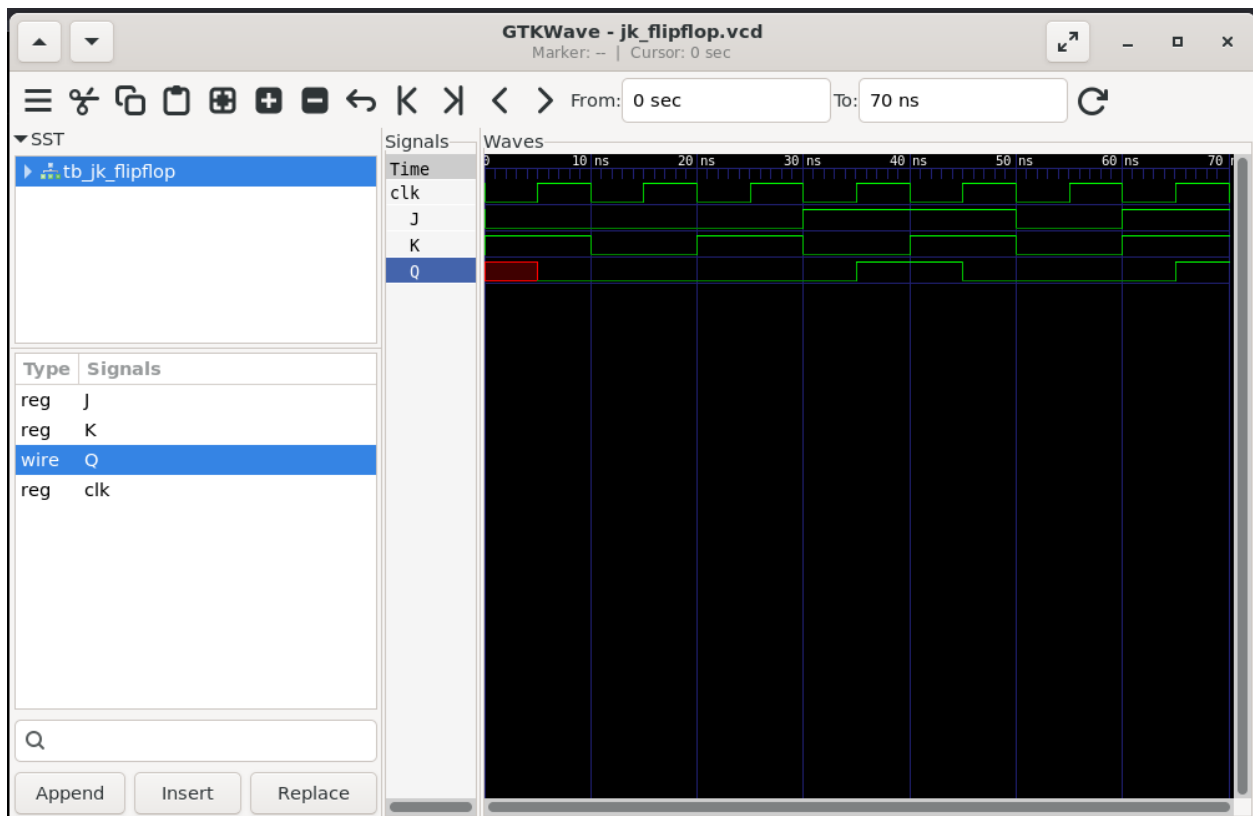
        #10 J = 1; K = 1;

        #10;

        $finish;
end
endmodule

```

3.3 Waveform Output



Observation:

The JK Flip-Flop was successfully simulated using Icarus Verilog and GTKWave. The waveform confirms the correct implementation of Hold, Reset, Set, and Toggle operations according to the J and K input combinations. The output changes only on the positive edge of the clock, validating the sequential behavior of the design.

4. Four-bit Register Implementation

4.1 Verilog Code

The following Verilog HDL code implements a 4-bit Register.

register4.v

```
module register4(  
  
    input clk,  
  
    input [3:0] D,  
  
    output reg [3:0] Q  
  
);  
  
always @(posedge clk)  
  
begin  
  
    Q <= D;  
  
end  
  
endmodule
```

4.2 Testbench Code

The following testbench was developed to verify the functionality of the 4-bit Register by applying different 4-bit input values. The testbench generates a periodic clock signal and applies various binary data patterns to the input. The output is observed on each positive edge of the clock to ensure that the register correctly stores and transfers the input data.

tb_register4.v

```
`timescale 1ns/1ps

module tb_register4;

    reg clk;

    reg [3:0] D;

    wire [3:0] Q;

    register4 uut (

        .clk(clk),

        .D(D),

        .Q(Q)

    );

    always # 5 clk = ~clk;

    initial

    begin

        $dumpfile("register4.vcd");

        $dumpvars(0, tb_register4);

        clk = 0;

        D =4'b0000;

        #10 D = 4'b1010;

        #10 D = 4'b1100;
```

```

#10 D = 4'b0111;

#10 D = 4'b1111;

#10 D = 4'b0011;

#10;

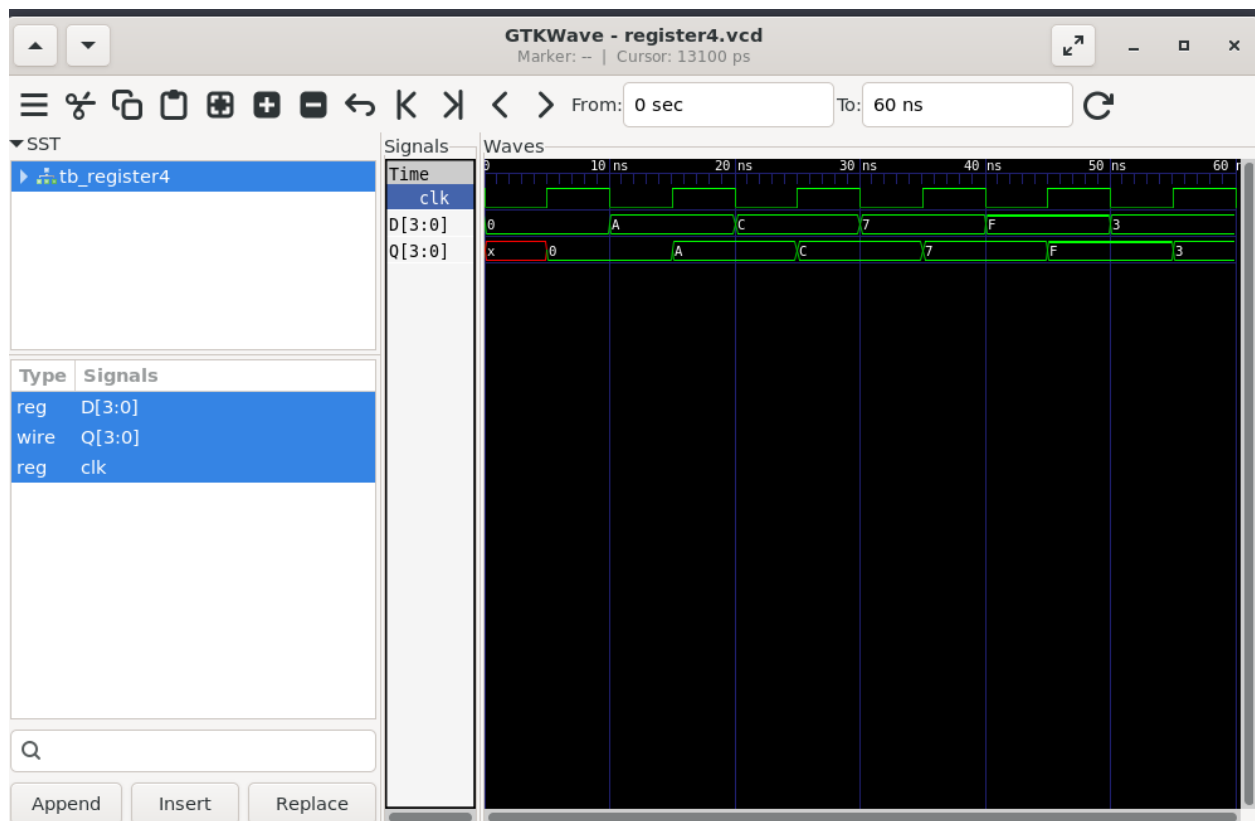
$finish;

end

endmodule

```

4.3 Waveform Output



Observation:

The 4-bit Register successfully stores and updates the input data on every positive edge of the clock. The waveform confirms that the output accurately follows the applied 4-bit input values, demonstrating correct sequential storage behavior.

5. Four-bit Binary Counter Implementation

5.1 Verilog Code

The following Verilog HDL code implements a 4-bit Binary Counter.

counter4.v

```
module counter4(  
    input clk,  
    output reg [3:0] Q  
);  
  
initial  
    Q = 4'b0000;  
  
always @(posedge clk)  
begin  
    Q <= Q + 1;  
end  
  
endmodule
```

5.2 Testbench Code

The following testbench was developed to verify the operation of the 4-bit Binary Counter. A periodic clock signal is generated to drive the counter, and the simulation is allowed to run for multiple clock cycles. The counter output is observed after each positive clock edge to ensure that it increments sequentially from 0000 to 1111 and then rolls over back to 0000.

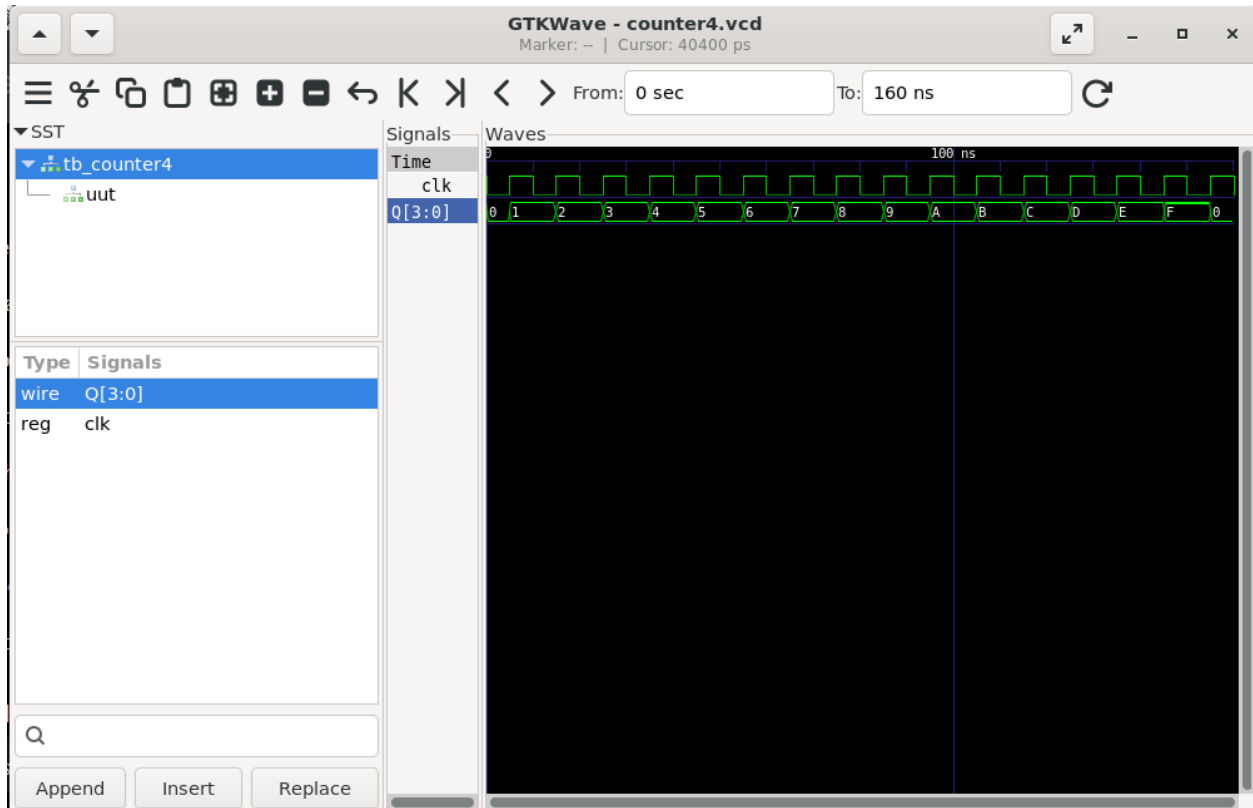
tb_counter4.v

```
`timescale 1ns/1ps
module tb_counter4;
    reg clk;
    wire [3:0] Q;
    counter4 uut (
        .clk(clk),
        .Q(Q)
    );
    always #5 clk = ~clk;
    initial
    begin
        $dumpfile("counter4.vcd");
        $dumpvars(0, tb_counter4);
        [
            clk = 0;

            #160;

            $finish;
        ]
    end
endmodule
```

5.3 Waveform Output



Observation:

The 4-bit Binary Counter successfully increments its output by one on every positive edge of the clock. The waveform shows the counting sequence progressing from **0000 (0)** to **1111 (15)** before rolling over to **0000**, confirming the correct operation of the counter.

6. Results

The D Flip-Flop, JK Flip-Flop, 4-bit Register, and 4-bit Binary Counter were successfully implemented using Verilog HDL. Functional verification was performed using dedicated testbenches, and waveform analysis using GTKWave confirmed that all sequential circuits behaved according to their expected functionality.

7. Applications

- FPGA Design
- ASIC Design
- Digital Registers
- Digital Counters
- Memory Elements
- Processor Design
- Embedded Systems
- VLSI Design and Verification

8. Conclusion

The implementation and verification of sequential circuits using Verilog HDL were successfully completed. Simulation with Icarus Verilog and waveform analysis using GTKWave verified the correctness of all implemented designs. This task provided practical experience in sequential logic design, HDL coding, simulation, and verification, which are essential components of the VLSI design flow.

■ References

1. Icarus Verilog Documentation
2. GTKWave User Guide
3. Verilog HDL – Samir Palnitkar
4. Digital Design – Morris Mano
5. IEEE Verilog HDL Standard